Resource-Aware FPGA Architecture for LSTM Edge Inference

Singhvi Prerit Rajneesh, NTU, School of Electrical and Electronic Engineering
Prof. Chang Chip Hong, NTU, School of Electrical and Electronic Engineering
Dr. Wang Zhou, Imperial Global Singapore
Dr. Vivek Mohan, Imperial Global Singapore

Abstract— Long Short-Term Memory (LSTM) networks are widely used for sequential data processing in edge applications, but their deployment on resource-constrained Field-Programmable Gate Arrays (FPGAs) is limited by the arithmetic cost of recurrent gate computation and nonlinear activation functions. This paper presents a resource-aware fixed-point FPGA architecture for scalar LSTM edge inference. Instead of evaluating the input, forget, candidate, and output gates using fully parallel datapaths, the proposed design reuses a shared Q4.12 fixed-point Multiply-Accumulate (MAC) processing path under Finite State Machine (FSM) control. The architecture integrates a widened accumulator with saturation logic, a piecewise polynomial activation unit supporting both sigmoid and tanh operations, a cell-level controller for micro-sequenced gate execution, and a parameterized sequence controller for multi-timestep inference. The design was implemented in SystemVerilog and verified through functional simulation across sequence lengths of two, three, and four timesteps. Regression tests covering nominal execution, varying inputs, recurrent feedback, signed fixed-point behavior, saturation-oriented cases, and held-start protocol behavior all produced the expected final hidden and cell states. Cycle-count measurements show deterministic latency scaling with sequence length, reflecting the expected area-latency tradeoff of the time-multiplexed approach. The results demonstrate the functional viability of a compact, micro-sequenced LSTM inference engine for edge-oriented FPGA deployment, with future work targeting synthesis, resource evaluation, vectorization, BRAM integration, and AXI-based system interfacing.

1. INTRODUCTION

Data processing is increasingly shifting from cloud computing to edge computing to leverage lower latency, higher bandwidth efficiency, and improved data privacy. One application in this domain is the edge deployment of Long Short-Term Memory (LSTM) networks, which are effective for sequential data modeling [1]. LSTMs are widely used in biomedical and sensor-based applications because of their ability to store and process temporal information, including data obtained from edge devices such as wearable monitors and implantable medical devices. However, deploying LSTM inference on edge Field-Programmable Gate Arrays (FPGAs) presents significant hardware challenges. LSTM architectures are computationally expensive due to complex data dependencies, nonlinear activation functions, and the computation of multiple recurrent gates at every timestep [2].

Directly mapping these operations to parallel arithmetic units can exhaust available Digital Signal Processing (DSP) slices and logic resources, resulting in an unfavorable area footprint for resource-constrained edge devices. While fixed-point arithmetic reduces the baseline resource consumption of these networks [3], structural optimizations are required to further minimize the arithmetic datapath. Recent accelerator designs address this by replacing parallel compute engines with time-multiplexed architectures that share hardware resources across operations [4].

This project investigates a resource-aware FPGA architecture for LSTM inference. Instead of computing the input, forget, candidate, and output gates in parallel, the proposed design reuses a shared fixed-point Multiply-Accumulate (MAC) processing path controlled by a Finite State Machine (FSM). The main contribution is a parameterized, micro-sequenced LSTM execution engine implemented in SystemVerilog. This design integrates a cell-level controller for gate sequencing with a sequence-level controller that manages execution across multiple timesteps. The resulting architecture maintains recurrent hidden and cell state feedback while minimizing the required arithmetic datapath, making it suitable for resource-constrained edge deployment.

2. BACKGROUND

2.1 Long Short-Term Memory Networks

Long Short-Term Memory (LSTM) networks are a variation of recurrent neural networks designed to process sequential data while mitigating the vanishing gradient problem inherent in conventional recurrent architectures [5]. An LSTM cell maintains two temporal states: the hidden state h_t, which serves as the output to subsequent layers or timesteps, and the cell state c_t, which acts as an internal memory path for long-term dependencies.

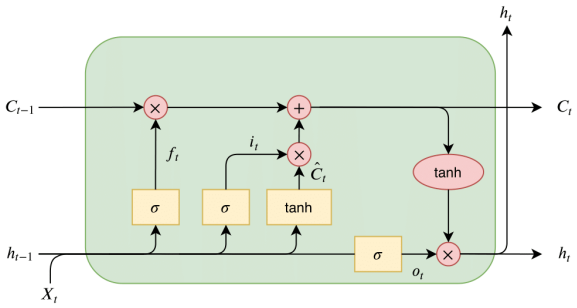

Figure 1: Standard LSTM cell structure.

At each timestep, an LSTM cell regulates the flow of information through four distinct gates: the input gate i_t, forget gate f_t, candidate gate g_t, and output gate o_t. The standard LSTM computations are defined as:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)\,, \tag{1}$$
$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)\,, \tag{2}$$
$$g_t = \tanh(W_g x_t + U_g h_{t-1} + b_g)\,, \tag{3}$$
$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)\,, \tag{4}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \tag{5}$$
$$h_t = o_t \odot \tanh(c_t)\,. \tag{6}$$

where x_t is the input, W and U denote input and recurrent weight parameters, b represents bias terms, $\sigma(\cdot)$ is the sigmoid activation function, $\tanh(\cdot)$ is the hyperbolic tangent

activation function, and $\odot$ denotes element-wise multiplication. Executing these equations requires repeated multiply-accumulate (MAC) operations, nonlinear function evaluations, and recurrent state feedback, creating significant computational challenges for hardware deployment.

2.2 FPGA-Based Edge Inference

Field-Programmable Gate Arrays (FPGAs) are effective platforms for edge inference due to their reconfigurability, spatial parallelism, and deterministic low latency [6]. Unlike Graphics Processing Units (GPUs), which are often optimized for high-throughput batch processing, FPGAs can be customized to execute application-specific datapaths for batch-size-one streaming workloads. This architectural flexibility is useful for low-latency embedded applications where sensor data must be processed close to its source.

However, deploying LSTM inference on resource-constrained edge FPGAs is challenging. A fully unrolled, parallel implementation of the four gate computations requires multiple DSP slices, activation units, and storage resources. For edge devices, the area-latency tradeoff must therefore be carefully optimized.

2.3 Hardware Optimization: Fixed-Point Datapaths and Time-Multiplexing

To reduce hardware overhead, floating-point arithmetic is commonly replaced with fixed-point quantization. By representing values with a fixed number of integer and fractional bits, MAC operations can be implemented with fewer logic resources and DSP slices [3]. This architecture uses signed $\mathrm{Q}4.12$ fixed-point arithmetic, where the 16-bit word contains one sign bit, three integer magnitude bits, and twelve fractional bits. Under this scheme, the value 1.0 is represented as 4096.

To reduce overflow during gate pre-activation accumulation, the architecture employs a widened accumulator for intermediate MAC results, followed by saturation logic when narrowing results back to the 16-bit datapath. Furthermore, computing nonlinear activation functions such as sigmoid and tanh directly in hardware can consume significant resources. Therefore, this design uses piecewise cubic polynomial approximation for the sigmoid function. The polynomial coefficients were generated offline using the Remez exchange algorithm and quantized to the $\mathrm{Q}4.12$ fixed-point format before being hardcoded into the RTL. The $\tanh$ function is then derived from the sigmoid approximation using

$$\tanh(x) = 2\sigma(2x) - 1,$$

allowing the same approximation hardware to support both activation functions.

Finally, the arithmetic footprint is reduced through time-multiplexing. Instead of instantiating parallel MAC units for all gates simultaneously, a single shared processing path is reused across the i_t, f_t, g_t, and o_t computations under Finite State Machine (FSM) control. This micro-sequenced approach trades execution latency for reduced hardware usage, aligning well with resource-constrained edge deployment.

3. PROPOSED ARCHITECTURE

3.1 System Overview

The proposed architecture implements a resource-aware scalar LSTM inference engine optimized for FPGA deployment. The design is organized around a reusable fixed-point processing core, a nonlinear activation unit, an LSTM cell datapath, a cell-level control Finite State Machine (FSM), and a parameterized sequence controller. Instead of computing all four LSTM gates in parallel, the architecture evaluates them sequentially using a shared processing path. This design choice reduces the required arithmetic units while trading execution latency for a compact area footprint.

The execution flow is divided into two levels of control. At the micro-architectural level, the cell controller sequences the operations required to compute a single timestep: input-weight multiplication, recurrent-weight multiplication, bias addition, activation evaluation, and state update. At the macro-architectural level, the sequence controller repeatedly invokes the scalar LSTM cell over `NUM_STEPS` timesteps, feeding the output hidden and cell states from one timestep back into the next.

The top-level sequence input is provided through `x_seq[NUM_STEPS]`, where each entry represents the scalar input at a given timestep. Initial hidden and cell states are supplied through `h_init` and `c_init`. Upon completion of all timesteps, the final hidden and cell states are exposed as `h_final` and `c_final`.

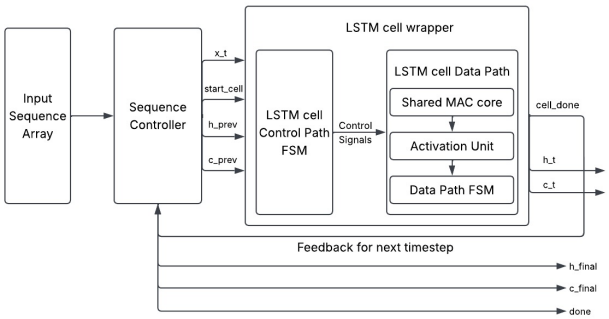

Figure 2: Top-level architecture of the proposed LSTM inference engine.

3.2 Fixed-Point Processing Core

The arithmetic core is built around a sequential Multiply-Accumulate (MAC) datapath using signed $\mathrm{Q}4.12$ fixed-point arithmetic. The design uses a 16-bit data path and a 48-bit accumulator. For each multiplication operation, the product is shifted right by the 12-bit fractional width to restore the correct Q-format scaling. Accumulated results are stored in the 48-bit accumulator to reduce the risk of intermediate overflow during sequential gate computations.

The processing core consists of the MAC module and a saturation checker. The MAC module performs fixed-point multiplication and accumulation under enable and clear control signals. The clear signal starts a new gate pre-activation computation, while the enable signal accumulates subsequent terms. After accumulation, the saturation checker narrows the 48-bit result back to the 16-bit datapath. If the accumulated value exceeds the representable signed 16-bit range, it is clamped to the maximum or minimum representable value, preventing wrap-around distortion.

3.3 Activation Function Unit

The activation function unit supports both sigmoid and hyperbolic tangent operations. Because direct computation of nonlinear functions such as sigmoid and $\tanh$ can consume significant logic resources on an FPGA, the sigmoid function is implemented using a segmented piecewise cubic polynomial approximation. The input domain is partitioned based on absolute input magnitude, with each region using a different set of quantized polynomial coefficients.

Piecewise polynomial approximation with quantization-aware coefficient selection is commonly used to reduce the hardware cost of nonlinear function implementation [7]. In this design, the polynomial coefficients were generated offline using the Remez exchange algorithm and quantized to the Q4.12 fixed-point format before being hardcoded into the RTL. To reduce duplicated hardware, $\tanh$ is not implemented as an independent approximation path. Instead, the identity

$$\tanh(x) = 2\sigma(2x) - 1$$

is used. In sigmoid mode, the unit outputs $\sigma(x)$. In $\tanh$ mode, the input is scaled by a factor of two, passed through the existing sigmoid approximation path, and reconstructed into the $\tanh$ output through constant scaling and subtraction. This method supports all nonlinear operations required by the LSTM cell while reducing duplicated activation hardware.

3.4 LSTM Cell Datapath

The LSTM cell datapath computes a single scalar timestep. It receives the current scalar input x_t, the previous hidden state h_{prev}, the previous cell state c_{prev}, gate weights W_i, W_f, W_g, W_o, recurrent weights U_i, U_f, U_g, U_o, and bias terms b_i, b_f, b_g, b_o.

For each gate, the datapath computes the pre-activation term sequentially:

$$z_{\text{gate}} = W_{\text{gate}} x_t + U_{\text{gate}} h_{\text{prev}} + b_{\text{gate}}.$$

A gate-select signal routes the appropriate gate parameters into the shared processing core. Operand-selection logic determines whether the core receives W, U, or b on one operand, and x_t, h_{prev}, or the fixed-point constant 1.0 on the other operand. Once the pre-activation value is accumulated, it is routed through the activation unit. Sigmoid is used for the input gate i_t, forget gate f_t, and output gate o_t, while $\tanh$ is used for the candidate gate g_t.

The activated scalar values are latched into internal registers. After all four gates have been computed, the cell state and hidden state are updated as:

$$c_t = f_t c_{\text{prev}} + i_t g_t, \tag{7}$$

$$h_t = o_t \tanh(c_t)\,. \tag{8}$$

The finalized c_t and h_t values are then exposed to the wrapper and sequence controller.

3.5 Cell-Level Control FSM

The cell-level FSM controls the micro-operations required for a single LSTM timestep. It manages gate selection, operand routing, processor enable and clear signals, pre-activation loading, gate-register loading, and final state loading. Each gate is evaluated through a fixed state progression: compute Wx_t, accumulate Uh_{prev}, add the bias term, latch the pre-activation, apply the nonlinear activation, and store the resulting gate output.

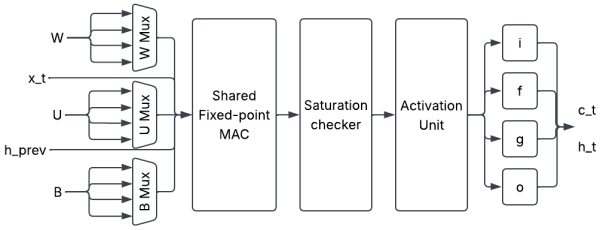

Figure 3: Shared MAC-based LSTM cell datapath.

After the input, forget, candidate, and output gates are processed, the FSM asserts load signals to finalize the cell state and hidden state updates. A done signal is then asserted to indicate that the timestep result is valid. This signal remains high as long as the initial start request is held, preventing accidental re-triggering before the request is released.

3.6 Parameterized Sequence Controller

The sequence controller extends the scalar LSTM cell to support multi-timestep inference. Parameterized by `NUM_STEPS`, it accepts the full sequence `x_seq[NUM_STEPS]`. Upon receiving a start assertion, the controller initializes the internal state registers with `h_init` and `c_init`. It then selects `x_seq[step_count]` as the current input and triggers the wrapped LSTM cell.

When the cell asserts done, the sequence controller latches the new h_t and c_t values, using them as h_{prev} and c_{prev} for the next timestep. The step counter increments, and the loop repeats until `NUM_STEPS` timesteps are executed. After the final timestep, the controller exposes `h_final` and `c_final`, asserting a global done flag until start is deasserted. This protocol ensures that only one complete sequence is executed per start assertion.

4. IMPLEMENTATION AND VERIFICATION

4.1 RTL Implementation

The proposed architecture was implemented in SystemVerilog using a modular Register Transfer Level (RTL) design methodology. The hierarchy is partitioned into arithmetic, activation, cell-level, and sequence-level modules. The arithmetic datapath consists of a MAC unit, a saturation checker, and a processor wrapper. The nonlinear activation function is isolated within a dedicated unit supporting both sigmoid and $\tanh$ modes. The LSTM cell datapath contains the gate-selection logic, operand-selection logic, intermediate gate registers, and final state-update logic.

To maintain a clear design hierarchy, the control logic is separated from the datapath. The cell-level controller, `LSTM_cell_cp.sv`, generates the micro-operation control signals required to execute a single LSTM timestep, including processor enable and clear signals, operand and gate selection signals, and register load signals. The integrated LSTM cell module encapsulates this datapath and controller into a single timestep execution unit. Finally, the sequence controller instantiates this unified cell and executes it iteratively over a parameterized number of timesteps.

Table 1: Main RTL modules.

Module	Function
MAC.sv	Performs sequential fixed-point multiply-accumulate operations.
Saturation_checker.sv	Narrows the 48-bit accumulator output to 16 bits with saturation logic.
Processor_top.sv	Wraps the MAC and saturation checker into a reusable arithmetic core.
ActFn.sv	Implements sigmoid and tanh using piecewise polynomial approximation.
LSTM_cell_dp.sv	Implements the scalar LSTM datapath and gate/state registers.
LSTM_cell_cp.sv	Controls micro-operation sequencing for one timestep.
LSTM_cell.sv	Integrates the datapath and control path.
LSTM_seq_ctrl.sv	Executes the scalar cell over NUM_STEPS timesteps.

4.2 Sequence Parameterization

The sequence controller is parameterized through the NUM_STEPS constant and accepts the input sequence through x_seq[NUM_STEPS]. This allows the same controller structure to be evaluated for different sequence lengths without modifying the underlying FSM. During execution, the controller selects x_seq[step_count], triggers the LSTM cell, and waits for the cell-level done signal. It then latches the resulting h_t and c_t outputs, using them as h_{prev} and c_{prev} for the subsequent timestep.

This parameterization was validated using testbench instances configured for NUM_STEPS $= 2$, NUM_STEPS $= 3$, and NUM_STEPS $= 4$. These configurations verify that the controller advances through the input array, preserves recurrent feedback between timesteps, and terminates after the configured number of steps.

4.3 Verification Methodology

Functional verification was performed using SystemVerilog testbenches. The sequence-controller testbench instantiates three versions of the design with NUM_STEPS $= 2$, NUM_STEPS $= 3$, and NUM_STEPS $= 4$. Each instance is subjected to the same regression suite, ensuring that the parameterized behavior remains valid across multiple sequence lengths.

The testbench drives fixed-point inputs, gate weights, recurrent weights, and bias values in signed Q4.12 format. Upon receiving the top-level done signal, the testbench compares the final state outputs h_final and c_final against expected golden values. The verification environment checks for unknown values, execution timeouts, correct step-count termination, and start/done protocol behavior. In the held-start scenario, the controller is expected to remain in the WAIT_START_LOW state until start is deasserted.

4.4 Test Cases

The verification suite is designed to exercise both datapath behavior and FSM state transitions. The nominal same-input case tests basic multi-step execution using a static input sequence, while the varying-input case confirms that x_seq[step_count] correctly advances through the input array. The recurrent-weight case isolates the hidden-state feedback path, ensuring that prior timesteps influence subsequent computations. Arithmetic robustness is verified through the bias and mixed-sign case, which checks signed fixed-point behavior, and the near-saturation case, which checks large-value behavior without wraparound. Finally, the held-start case verifies that the controller does not retrigger a new sequence until start is released.

Table 2: Sequence-controller regression cases.

Test Case	Verification Target
Nominal same input	Basic multi-step sequence execution.
Nominal varying input	Correct selection of x_seq[step_count].
Nonzero recurrent U	Hidden-state feedback across timesteps.
Nonzero bias and mixed input	Bias handling and signed fixed-point arithmetic.
Near saturation	Saturation behavior and avoidance of integer wraparound.
Held-start edge case	Start/done protocol and WAIT_START_LOW behavior.

4.5 Evaluation Metrics

The architecture is evaluated using five primary metrics:

Functional correctness: It is determined by comparing the final cell state c_final and hidden state h_final against golden expected values for every regression case and sequence length. This verifies that the arithmetic core, activation unit, recurrent feedback path, and sequence controller agree with the expected fixed-point LSTM behavior.

Cycle count: It is measured as the total number of clock cycles between start assertion and top-level done assertion. This captures the latency cost of the time-multiplexed architecture. Because a single MAC path is reused across all four gates, latency is expected to scale approximately linearly with NUM_STEPS.

Protocol correctness: It verifies the start/done control behavior. Specifically, the held-start test ensures that the controller remains in WAIT_START_LOW after execution and does not begin another sequence until start is deasserted.

Parameterized sequence correctness: It ensures that the controller traverses x_seq correctly, preserves temporal dependencies through h_t and c_t feedback, and halts at the configured NUM_STEPS value.

Numerical robustness: It is evaluated using unknown-value checks, timeout limits, and saturation-oriented test cases. These checks confirm that the Q4.12 datapath remains stable and avoids wraparound behavior during large fixed-point accumulations.

5. RESULTS AND DISCUSSION

5.1 Functional Verification Results

The parameterized sequence controller was verified for NUM_STEPS $= 2$, NUM_STEPS $= 3$, and NUM_STEPS $= 4$ using the regression cases defined in Section 4. Each configuration executed successfully and produced the expected fixed-point final cell and hidden states. The simulation confirms that the sequence controller correctly traverses x_seq[step_count], maintains recurrent h_t and c_t feedback across timesteps, and terminates at the configured sequence boundary.

5.2 Sequence-Length Scaling

Simulation confirms that the controller structure scales deterministically across the tested NUM_STEPS configurations. In the nominal same-input case, c_final increases from 2942 for NUM_STEPS $= 2$ to 3968 for NUM_STEPS $= 4$, reflecting recurrent state accumulation during repeated sequence execution. The nominal varying-input case exhibits decreas-

Table 3: Functional verification results.

NUM_STEPS	Test Case	c_final (Int)	h_final (Int)	Status
2	Nominal same input	2942	1842	Pass
3	Nominal same input	3535	2092	Pass
4	Nominal same input	3968	2243	Pass
2	Nominal varying input	1949	1125	Pass
3	Nominal varying input	1380	746	Pass
4	Nominal varying input	866	452	Pass
2	Nonzero recurrent U	2885	1577	Pass
3	Nonzero recurrent U	2209	1165	Pass
4	Nonzero recurrent U	1548	812	Pass
2	Nonzero bias and mixed input	475	206	Pass
3	Nonzero bias and mixed input	652	312	Pass
4	Nonzero bias and mixed input	413	185	Pass
2	Large value accumulation	12288	4096	Pass
3	Large value accumulation	16384	4096	Pass
4	Large value accumulation	20480	4096	Pass
2	Held-start edge case	1949	1125	Pass
3	Held-start edge case	1380	746	Pass
4	Held-start edge case	866	452	Pass

ing final state magnitudes as later inputs become smaller, confirming that the `x_seq[step_count]` selection logic advances correctly rather than repeatedly using the first input. Furthermore, the recurrent-weight test verifies that the hidden-state feedback path is active, since the output of timestep $t-1$ influences the gate pre-activation of timestep t through the recurrent U weights.

5.3 Latency and Time-Multiplexing Tradeoff

The architecture intentionally time-multiplexes a single MAC datapath across all four LSTM gates. This reduces arithmetic resource usage at the cost of execution latency, which scales linearly across the tested `NUM_STEPS` values.

The deterministic cycle counts obtained from functional simulation are shown in Table 4. The cycle count is measured from start assertion to top-level done assertion. Since the FSM schedule is data-independent, the same cycle count is expected for all regression cases with the same `NUM_STEPS`. The held-start protocol check occurs after done assertion and is not included in the latency count.

Table 4: Cycle-count latency.

NUM_STEPS	Test Case	Total Clock Cycles
2	Nominal same input	53
3	Nominal same input	79
4	Nominal same input	105

These measurements quantify the latency cost of the time-multiplexed schedule. A more parallel LSTM implementation could reduce latency by evaluating multiple gates simultaneously, but would require additional MAC units, activation paths, and routing resources. In contrast, the proposed design favors temporal reuse, making it more appropriate for area-constrained edge FPGA scenarios.

5.4 Discussion and Limitations

The RTL implementation demonstrates multi-timestep execution for a scalar LSTM engine. Validated capabilities include sequential input traversal, $\mathrm{Q}4.12$ signed fixed-point behavior for the tested cases, piecewise polynomial activation evaluation, recurrent state feedback, and synchronous start/done FSM protocol behavior.

However, functional simulation alone is insufficient to fully prove hardware efficiency. To validate the resource-aware claim quantitatively, the RTL should be synthesized for an edge-class FPGA device, such as a Basys3 or ZedBoard, to extract LUT, flip-flop, DSP slice, block RAM, and power utilization metrics. Furthermore, the current design is a scalar prototype. Extending the architecture to support vectorized input data, BRAM-based weight fetching, and standard accelerator interfaces such as AXI would be necessary for deployment as a complete system-level accelerator.

6. CONCLUSION AND FUTURE WORK

This work presented a resource-aware, micro-sequenced fixed-point LSTM engine tailored for FPGA-based edge inference. To address the logic and arithmetic resource constraints of edge devices, the architecture uses a shared Multiply-Accumulate (MAC) datapath governed by a cell-level Finite State Machine (FSM), trading execution latency for a compact arithmetic footprint. The design integrates signed $\mathrm{Q}4.12$ fixed-point arithmetic, saturation-aware accumulation, and piecewise polynomial activation approximation to execute LSTM gate operations without requiring fully parallel gate datapaths.

To support temporal data processing, a parameterized sequence controller was designed to extend the scalar LSTM cell across multi-timestep inference. Functional simulation validated the controller's ability to traverse input sequences through `x_seq[NUM_STEPS]`, preserve recurrent hidden and cell state feedback through h_t and c_t, and terminate at configurable sequence boundaries. Verification across `NUM_STEPS` $= 2$, `NUM_STEPS` $= 3$, and `NUM_STEPS` $= 4$ confirmed correct behavior of the time-multiplexed datapath across nominal, recurrent, signed-input, saturation-oriented, and held-start protocol test cases.

Overall, this scalar prototype demonstrates the viability of time-multiplexed LSTM computation for resource-constrained FPGA inference. By reusing arithmetic hardware across gates and timesteps, the design captures the key area-latency tradeoff required for compact edge-oriented recurrent neural network accelerators.

While functional correctness has been verified, future work will focus on synthesizing the RTL on an edge-class FPGA to extract Look-Up Table (LUT), Flip-Flop (FF), DSP slice, Block RAM (BRAM), timing, and power utilization metrics. Subsequent architectural extensions will focus on transforming this scalar core into a deployable system-level accelerator.

This includes vectorizing the datapath for multi-element operations, integrating BRAM for memory-backed weight and input buffering, and wrapping the sequence controller with a standard Advanced eXtensible Interface (AXI) to enable integration with embedded processors.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Professor Chang Chip Hong for the opportunity to join this project and for his valuable guidance throughout the work. I am also deeply grateful to Dr. Wang Zhou and Dr. Vivek Mohan for their continuous support and feedback during the project. I would like to acknowledge the funding support from the Nanyang Technological University URECA Undergraduate Research Programme.

REFERENCES

[1] C. Gao, A. Rios-Navarro, X. Chen, S.-C. Liu, and T. Delbruck, "EdgeDRNN: Recurrent Neural Network Accelerator for Edge Inference," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 10, no. 4, pp. 419–432, 2020. https://doi.org/10.1109/JETCAS.2020.3040300

[2] D. He, J. He, J. Liu, J. Yang, Q. Yan, and Y. Yang, "An FPGA-Based LSTM Acceleration Engine for Deep Learning Frameworks," *Electronics*, vol. 10, no. 6, p. 681, 2021. https://doi.org/10.3390/electronics10060681

[3] S.-E. Chang, Y. Li, M. Sun, R. Shi, H. K.-H. So, X. Qian, Y. Wang, and X. Lin, "Mix and Match: A Novel FPGA-Centric Deep Neural Network Quantization Framework," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 208–220, 2021. https://doi.org/10.1109/HPCA51647.2021.00027

[4] N. Mao, H. Yang, and Z. Huang, "An Instruction-Driven Batch-Based High-Performance Resource-Efficient LSTM Accelerator on FPGA," *Electronics*, vol. 12, no. 7, p. 1731, 2023. https://doi.org/10.3390/electronics12071731

[5] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. https://doi.org/10.1162/neco.1997.9.8.1735

[6] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, "A Survey of FPGA-Based Neural Network Inference Accelerators," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 12, no. 1, Article 2, 2019. https://doi.org/10.1145/3289185

[7] H. Geng, X. Chen, N. Zhao, Y. Du, and L. Du, "QPA: A Quantization-Aware Piecewise Polynomial Approximation Methodology for Hardware-Efficient Implementations," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 7, pp. 931–944, 2023. https://doi.org/10.1109/TVLSI.2023.3277023